

Nama : Carlos Fadellilah Rama

Kelas : TI-1H

Matkul : Prak. ASD

Praktikum 1

1. Apa maksud potongan kode berikut pada method enqueue?

Java

```
if (rear == max - 1) {  
    rear = 0;  
}
```

Jawaban:

Potongan kode tersebut digunakan untuk mengimplementasikan **Circular Queue** (antrian melingkar). Jika posisi rear (indeks belakang) sudah mencapai batas maksimal array ($\text{max} - 1$), namun antrian belum penuh (karena sudah ada data yang di-dequeue), maka nilai rear akan dikembalikan ke indeks **0**. Dengan cara ini, ruang kosong di bagian depan array dapat digunakan kembali.

2. Apa maksud potongan kode berikut pada method dequeue?

Java

```
if (front == max - 1) {  
    front = 0;  
}
```

Jawaban:

Sama halnya dengan enqueue, potongan kode ini juga merupakan bagian dari logika **Circular Queue**. Jika posisi front (indeks depan) sudah mencapai batas akhir array dan ada proses penghapusan data (dequeue), maka posisi front akan berpindah kembali ke indeks **0**. Ini memastikan antrian tetap bisa berjalan terus melingkar selama masih ada elemen di dalamnya.

3. Pada method print, mengapa pada proses perulangan variabel i tidak dimulai dari 0 (int i = 0), melainkan int i = front?

Jawaban:

Karena pada struktur data Queue dengan array, elemen terdepan (data pertama dalam antrian) tidak selalu berada di indeks 0. Posisi data terdepan ditunjukkan oleh variabel front. Jika kita memulai perulangan dari 0, data yang ditampilkan mungkin bukan urutan antrian yang benar atau bahkan bisa menampilkan data sampah (data yang sudah dihapus secara logika).

4. Perhatikan method print, jelaskan maksud dari potongan kode berikut!

Java

```
i = (i + 1) % max;
```

Jawaban:

Potongan kode ini berfungsi untuk menggeser indeks *i* ke posisi berikutnya secara **modulus**. Tujuannya adalah agar ketika indeks *i* sudah mencapai batas maksimal (*max* - 1), perhitungan $(i + 1) \% \text{max}$ akan menghasilkan nilai **0**. Ini memungkinkan proses cetak data mengikuti alur melingkar dari *Circular Queue*.

5. Tunjukkan potongan kode pada method print yang berfungsi untuk menampilkan data yang dilakukan sebanyak size!

Jawaban:

Potongan kode yang mengatur jumlah data yang ditampilkan agar sesuai dengan jumlah elemen yang ada (*size*) adalah penggunaan struktur perulangan *while* dan penanda akhir setelah loop. Secara spesifik:

Java

```
int i = front;
```

```
while (i != rear) {
```

```
    System.out.print(data[i] + " ");
```

```
    i = (i + 1) % max;
```

```
}
```

```
System.out.println(data[i] + " "); // Menampilkan elemen terakhir (posisi rear)
```

Di sini, perulangan terus berjalan dari *front* hingga mencapai *rear*. Karena kondisi *i* != *rear* berhenti sebelum mencetak elemen terakhir, maka ditambah satu baris cetak lagi di luar loop untuk elemen di posisi *rear*.

6. Apa yang dimaksud dengan *Queue Overflow*? Pada method mana kejadian tersebut terjadi?

Jawaban:

- **Definisi:** *Queue Overflow* adalah kondisi di mana kita mencoba memasukkan data baru ke dalam antrian, namun antrian tersebut sudah penuh (mencapai kapasitas maksimal).
- **Lokasi:** Kejadian ini terjadi pada method **enqueue**. Di dalam kode, hal ini ditandai dengan pengecekan *if (isFull())*.

7. Apa yang dimaksud dengan *Queue Underflow*? Pada method mana kejadian tersebut terjadi?

Jawaban:

- **Definisi:** *Queue Underflow* adalah kondisi di mana kita mencoba mengambil atau menghapus data dari antrian, namun antrian tersebut dalam keadaan kosong.
- **Lokasi:** Kejadian ini terjadi pada method **dequeue** dan juga bisa terjadi pada method **peek**. Di dalam kode, hal ini ditandai dengan pengecekan if (isEmpty()).

Praktikum 2

1. Modifikasi Class AntrianLayanan

Tambahkan method LihatAkhir di dalam class ini untuk melihat data mahasiswa yang berada di posisi paling belakang antrian.

Java

```
public void LihatAkhir() {  
    if (!isEmpty()) {  
        System.out.println("Antrian paling belakang: " +  
            data[rear].nim + " " + data[rear].nama + " " +  
            data[rear].prodi + " " + data[rear].kelas);  
    } else {  
        System.out.println("Antrian masih kosong");  
    }  
}
```

2. Modifikasi Class LayananAkademikSIKAD

Tambahkan opsi menu ke-5 (atau ke-6 sesuai petunjuk jobsheet) untuk memanggil method LihatAkhir.

Java

```
import java.util.Scanner;  
  
public class LayananAkademikSIKAD {  
    public static void menu() {  
        System.out.println("Pilih menu:");  
        System.out.println("1. Antrian baru");  
    }  
}
```

```

        System.out.println("2. Antrian keluar");
        System.out.println("3. Cek antrian terdepan");
        System.out.println("4. Cek semua antrian");
        System.out.println("5. Cek antrian paling belakang"); // Penambahan menu
        System.out.println("-----");
    }

```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Masukkan kapasitas antrian: ");
    int n = sc.nextInt();
    AntrianLayanan antri = new AntrianLayanan(n);

    int pilih;
    do {
        menu();
        pilih = sc.nextInt();
        sc.nextLine();
        switch (pilih) {
            case 1:
                System.out.print("NIM: ");
                String nim = sc.nextLine();
                System.out.print("Nama: ");
                String nama = sc.nextLine();
                System.out.print("Prodi: ");
                String prodi = sc.nextLine();
                System.out.print("Kelas: ");
                String kelas = sc.nextLine();
                Mahasiswa mhs = new Mahasiswa(nim, nama, prodi, kelas);

```

```

        antri.enqueue(mhs);
        break;
    case 2:
        Mahasiswa dataKeluar = antri.dequeue();
        if (dataKeluar != null) {
            System.out.println("Antrian keluar: " + dataKeluar.nim + " " +
dataKeluar.nama);
        }
        break;
    case 3:
        antri.peek();
        break;
    case 4:
        antri.print();
        break;
    case 5:
        antri.LihatAkhir(); // Memanggil method yang baru dibuat
        break;
    }
} while (pilih > 0 && pilih < 6);
}
}

```

Penjelasan Singkat:

- **Method LihatAkhir:** Menggunakan atribut rear sebagai indeks untuk mengakses elemen terakhir yang masuk ke dalam array data.
- **Logika Circular:** Meskipun antrian berputar (circular), atribut rear selalu menunjuk ke data yang paling baru dimasukkan, sehingga kita bisa langsung mengaksesnya selama antrian tidak kosong.
- **Buffer Flush:** Penggunaan `sc.nextLine()` setelah `sc.nextInt()` sangat penting agar input teks (String) berikutnya tidak terlewat oleh karakter *newline* yang tersisa.